VISUAL OS

The Governance Operating System for

Autonomous Physical Operations

Architecture Blueprint & MVP Roadmap

Nx CONFIDENTIAL | April 2026 | Version 1.0

Table of Contents

1. Executive Summary
2. Vision & Market Thesis
3. Architectural Principles
4. Core Architecture
5. Entity Model
6. World Model Layer
7. Coding Engine
8. Tiered Infrastructure
9. Security Considerations
10. MVP Definition
11. Development Roadmap
12. Resourcing and Dependencies
13. Competitive Positioning
Appendix A: Glossary

1. Executive Summary

The wave of autonomous physical operations — robotic housekeepers, automated warehouses, self-driving delivery fleets, AI-managed retail — is creating a critical infrastructure gap. Today, video monitoring is a passive recording system watched by humans. Tomorrow, when the things being monitored are themselves autonomous, we need a fundamentally different system: one that can define what's allowed, verify compliance in real time, provide situational context to autonomous agents, and maintain human oversight over an increasingly automated physical world.

Visual OS is that system. It is the governance and oversight operating system for autonomous physical operations, where camera and sensor infrastructure provides the perceptual substrate, but the actual product is a programmable, AI-native platform for defining, monitoring, and enforcing policy over physical spaces.

The Claude Code Analogy

Claude Code transformed software development by letting developers express intent and having the system translate that into precise operations on code entities (files, functions, tests, git repos). Visual OS does the same for physical-world operations: operators express governance intent, and the system translates it into operations on domain entities (streams, cameras, zones, storage tiers, processing pipelines, world models). The operator doesn't write Python — they program reality.

This document defines the architecture, entity model, world model layer, coding engine, tiered infrastructure, MVP scope, and four-phase roadmap for Visual OS. The MVP is structured around five "wow moments" that demonstrate capabilities no existing video monitoring system can match: the system models its own uncertainty, operators program reality through domain entities, multiple world models interpret the same scene from different governance perspectives, autonomous agents consume situational awareness through a context API, and the system degrades transparently across infrastructure tiers.

1.1 Relationship to Nx Meta

Visual OS builds on top of Network Optix's existing Nx Meta VMS platform rather than replacing it. Nx Meta provides the foundational video infrastructure layer — RTSP ingestion, device management, storage management, and the existing plugin ecosystem — that Visual OS extends with world model intelligence, the coding engine, and the context API. Existing Nx Meta customers gain an upgrade path to Visual OS capabilities without abandoning their current deployment. Components reused from Nx Meta (stream ingestion, device discovery, storage backends) are not rebuilt; the MVP development effort focuses exclusively on the new layers (world model, coding engine, policy engine, context API, and dashboard).

2. Vision & Market Thesis

2.1 The Problem

Current video monitoring systems are built as recording and viewing tools. They assume a human watches screens and makes decisions. This model breaks down when:

- **Autonomous agents operate physical spaces.** Robotic cleaners, delivery robots, automated forklifts, and AI-managed systems need continuous oversight that humans cannot provide at scale.
- **Policy becomes complex.** A factory with 50 robots, 200 cameras, and 3 shifts has thousands of rules about who can go where, when, and under what conditions. These rules change constantly and cannot be managed through static configuration.
- **Multiple systems need context.** Robots need to know if a room is occupied. Building management needs to know traffic patterns. Compliance needs audit trails. Security needs identity verification. No current system serves all these consumers from a unified model of reality.
- **Connectivity is unreliable.** Edge devices, facility servers, and cloud infrastructure operate at different reliability levels. The system must degrade gracefully, not fail completely, when connectivity drops.

2.2 The Opportunity

Just as every cloud deployment needs observability infrastructure (Datadog, Grafana, PagerDuty), every autonomous physical operation will need governance infrastructure. Visual OS occupies this layer — it is the observability and governance platform for the physical world.

The market timing is driven by three converging trends: the deployment of autonomous agents in commercial settings, the maturation of video and physical-world AI models that enable semantic scene understanding, and the increasing regulatory pressure for compliance and audit trails in automated operations.

2.3 The Strategic Bet

The system that becomes the standard governance layer for autonomous physical operations will own a critical piece of infrastructure. Visual OS is designed to be that system — built from day one as a platform that other systems depend on, not an application that humans watch.

3. Architectural Principles

Seven non-negotiable commitments that guide every design decision:

Principle 1: World model as center of gravity

Cameras and sensors are observers, not the system's entities. They produce observations that update a model of reality. The world model — the system's best current belief about the physical space — is the central data structure. Everything upstream feeds into it; everything downstream reads from it.

Principle 2: Multiple partial models, not one monolith

Reality is never fully observable. The system maintains multiple overlapping, purpose-built world models (security, operational, safety, compliance), each with its own coverage geometry, update cadence, error asymmetry, and confidence semantics. Uncertainty is a first-class citizen, not an error state. Every assertion carries a confidence score, a staleness timestamp, and a source attribution.

Principle 3: Entity-oriented coding engine

Operators program the system in terms of domain entities (streams, zones, policies, storage tiers, pipelines), not in Python or YAML. The coding engine understands the entity model, validates operations, manages permissions, and translates intent into executable operations — exactly as Claude Code translates coding intent into file operations.

Principle 4: Tiered infrastructure with graceful degradation

The system operates across three tiers — edge, facility, and cloud — each with different compute, storage, connectivity, and cost characteristics. When a tier becomes unavailable, the system degrades to reduced capabilities rather than failing. The world model is replicated across tiers with appropriate fidelity.

Principle 5: Permission-gated actions with physical-world safety

Every action the system can take (alert, command a robot, lock a door, escalate) is permission-gated with configurable approval workflows. Actions with physical consequences require higher confidence thresholds than digital actions. The system defaults to safe states when uncertain.

Principle 6: Platform, not application

Visual OS is infrastructure that other systems consume, not an application that humans watch. The context API serves situational awareness to autonomous agents. The integration protocol lets third-party systems plug in. The coding engine lets operators extend capabilities without modifying the core.

Principle 7: Interface stability, implementation evolution

The world model interface, entity model, and coding engine API are designed to be stable. The implementations behind them — the AI models, the processing pipelines, the scene

understanding algorithms — will evolve rapidly as video and physical-world models advance. Design the contracts now; upgrade the implementations continuously.

4. Core Architecture

The system is organized into seven layers, with the world model layer at the center. Layers below the world model feed observations into it; layers above consume the world model for decision-making and action.

4.1 Architecture Stack

Layer	Name	Responsibility
7 (top)	Human interface	Dashboard, NL queries, incident review, policy authoring
6	Coding engine	Translates operator intent into entity operations; the Claude Code analog
5	Policy engine	Continuous evaluation of governance rules against world model state
4	Context API	Serves world model state to autonomous agents and external systems
3 (center)	World model layer	Multiple partial, overlapping models of physical reality with confidence
2	Entity runtime	Manages streams, storage, pipelines, models, connectivity lifecycle
1 (bottom)	Observers	Cameras, sensors, robots, access systems, schedules, human input

4.2 Performance Budgets

End-to-end latency targets must be defined for each critical path through the architecture stack: observer → world model update, world model → policy evaluation, policy → action dispatch, and Context API query response. These budgets should be specified as p50/p95/p99 targets and validated through automated benchmarks. Performance budgets are a required deliverable alongside the MVP and will drive key architectural decisions (batch vs. streaming, polling vs. push, GPU sizing).

4.3 Design Inspiration: Claude Code Architecture

The architecture draws inspiration from the publicly documented design patterns of Claude Code and similar production-grade agentic systems. These systems demonstrate that a successful agentic platform requires: a central orchestration engine, a permission-gated tool system, a multi-tier memory system, multi-agent orchestration with coordinator/worker patterns, and a plugin/extension protocol (e.g., MCP) for external integration.

Visual OS maps these patterns onto the physical-world domain:

Claude Code Component	Visual OS Analog	Key Difference
QueryEngine (orchestration)	Reasoning engine + policy evaluator	Real-time continuous loop vs. request-response
Tool system (50+ tools)	Action system (alerts, robot commands, gates)	Physical consequences require confidence thresholds
CLAUDE.md + memory tiers	World model layer (spatial, temporal, semantic)	Multiple partial models with uncertainty vs. single memory
Permission system (3 modes)	Policy engine with approval workflows	Zone-based spatial policies + time-based rules
Multi-agent (subagents, swarms)	Agent orchestration (zone agents, supervisors)	Camera agents, robot supervisors, specialist agents
MCP / plugins / skills	Integration protocol	Robot APIs, BMS, access control, third-party analytics
Context compression (3 strategies)	Stream management + attention allocation	Video frame budget vs. token budget

5. Entity Model

The coding engine operates on domain-specific entities — the fundamental primitives of the visual monitoring domain. Each entity has typed properties, operations, constraints, and relationships to other entities.

5.1 Stream Entities

Stream

The foundational entity. A stream is a continuous flow of data from an observer (camera, sensor, robot telemetry). Streams have sources, codecs, resolutions, frame rates, current state, and a location in the tiered infrastructure.

- **Properties:** sourceId, codec, resolution, frameRate, state (live | buffered | archived), currentTier (edge | facility | cloud), bitrate, latencyMs, healthScore
- **Operations:** route(destination), downsample(resolution), process(pipeline), store(tier, retention), subscribe(eventFilter), pause(), resume()

- **Constraints:** bandwidth budget per tier, maximum concurrent streams per processor, storage cost per tier per hour

Camera

A programmable observer with onboard capabilities. Cameras are not just stream sources — they have PTZ control, onboard storage, edge AI capabilities, and health telemetry.

- **Properties:** model, location (3D coordinates), fieldOfView, capabilities (ptz | thermal | edgeAI | onboardStorage), status, firmwareVersion, coverageZones[]
- **Operations:** pan(degrees), tilt(degrees), zoom(level), setPreset(name), enableEdgeDetection(model), getHealth(), reboot()
- **Constraints:** PTZ speed limits, onboard storage capacity, edge inference FPS limits

Sensor

Non-video observers: motion detectors, door contacts, environmental sensors, badge readers, weight plates.

- **Properties:** type, location, triggerCondition, currentValue, lastTriggered, batteryLevel, coverageRadius
- **Operations:** subscribe(callback), calibrate(), setThreshold(value), getHistory(timeRange)

5.2 Infrastructure Entities

Storage Tier

Storage is not monolithic — it is a tiered entity with radically different characteristics at each level.

Tier	Location	Retention	Latency	Cost	Availability
Edge buffer	Camera / edge device	24–72 hours	< 1ms	Free (HW amortized)	Always (local)
Facility NVR	On-premise server	7–90 days	< 10ms	Fixed (server cost)	High (local network)
Cloud archive	Cloud object storage	1 year+	50–500ms	Per-GB monthly	Requires connectivity
Cloud hot	Cloud SSD	30 days	10–50ms	Higher per-GB	Requires connectivity

- **Operations:** store(stream, retention, evictionPolicy), retrieve(query, timeRange), migrate(fromTier, toTier), evict(policy), getCapacity(), getCost(timeRange)

Pipeline

A composable chain of processing steps applied to a stream. Pipelines are first-class entities that can be defined, versioned, and assigned to streams or zones.

- **Steps:** detect(model) → classify(model) → track(algorithm) → vectorize(embeddingModel) → index(vectorStore)
- **Properties:** steps[], targetTier (which tier runs each step), estimatedLatency, estimatedCostPerHour, requiredGPU
- **Operations:** deploy(stream), undeploy(), scale(replicas), getMetrics(), fallback(degradedPipeline)
- **Constraints:** Some steps only run at certain tiers; GPU availability; maximum processing budget

Model (ML)

A deployable inference model with capability metadata. The coding engine must understand what each model can do, where it can run, and what it costs.

- **Properties:** name, version, capabilities (detection | classification | tracking | embedding | NLQuery), requiredCompute (cpu | gpu | tpu), quantization, supportedTiers[], accuracyScore, latencyMs, costPerInference
- **Operations:** deploy(tier), undeploy(), infer(input), benchmark(), getMetrics()

Connectivity

Connectivity between tiers is itself a managed entity with state, health, and fallback behavior.

- **Properties:** fromTier, toTier, state (connected | degraded | offline), bandwidthMbps, latencyMs, lastConnected, uptimePercent
- **Operations:** healthCheck(), getCapacity(), onStateChange(callback)
- **Constraints:** When offline, queued operations persist and replay on reconnection; the coding engine must handle all three states

5.3 Governance Entities

Zone

A named spatial region with semantic properties and governance rules. Zones are the bridge between the spatial world model and the policy engine.

- **Properties:** name, boundary (polygon/3D volume), type (restricted | public | transitional | hazardous), schedule (when rules apply), observingCameras[], sensors[], currentOccupancy, confidenceScore

- **Operations:** definePolicy(rule), getOccupancy(), getHistory(timeRange), setAlertLevel(level), subdivide(subzones)

Policy

A governance rule that the system evaluates continuously against world model state.

- **Properties:** name, condition (predicate over world model), action (what to do when true), severity, confidenceThreshold (minimum confidence to trigger), cooldownSeconds, scope (zones[], timeWindow)
- **Operations:** enable(), disable(), evaluate(worldModelState), getHistory(), test(simulatedState)

Approval Workflow

For actions that require human confirmation before execution. Mirrors Claude Code's permission modes: auto-allow, ask-human, auto-deny.

- **Properties:** triggerCondition, approvers[], timeoutSeconds, defaultOnTimeout (allow | deny), escalationChain[]

6. World Model Layer

The world model is the architectural center of gravity. It is the system's continuously updated representation of physical reality — maintained through sensor fusion, enriched by AI inference, and consumed by every decision-making component.

> **Critical Design Decision: Partial Observability**
> The world model is never a perfect mirror of reality. It is the system's best current belief, always incomplete, always uncertain, and always potentially wrong. This is not a limitation to be fixed — it is a fundamental property to be designed for. Every assertion in the world model carries a confidence score, a staleness timestamp, and a source attribution. The system's reliability comes from knowing what it doesn't know.

6.1 Multiple Purpose-Built Models

Rather than one monolithic digital twin, the system maintains multiple overlapping models, each optimized for a different governance concern:

Model	Update Rate	Error Bias	Coverage Focus	Primary Consumers
Security	Real-time	Cautious (assume threat)	Access points, perimeter, restricted zones	Security team, access control, identity systems

Operational	Near-real-time (seconds)	Neutral (assume normal)	Production floor, docks, corridors	Ops manager, robot fleet, scheduling systems
Safety	Sub-second	Cautious (assume danger)	Hazard zones, machinery, exits	Safety system, emergency shutdown, alarms
Compliance	Batch (minutes)	Documented uncertainty	All monitored areas (retrospective)	Compliance officers, auditors, legal

6.2 World Model Sub-Components

Spatial Model

The physical layout of the monitored space. Evolution path:

- **Day 1:** 2D floor plan with named polygon zones, camera positions, and basic topology (which zone connects to which)
- **Phase 2:** 3D spatial mesh with object positions, camera frustums, and coverage calculation
- **Phase 3:** Semantic spatial graph with relationships ("this corridor connects zone A to zone B", "this door is the only exit from zone C")
- **Future:** Full physics-aware digital twin supporting simulation ("if we add a camera here, what coverage do we gain?")

Temporal Model

What is happening and what has happened. Evolution path:

- **Day 1:** Event log with timestamps and zone associations
- **Phase 2:** Pattern recognition ("this zone is normally empty between 22:00–06:00")
- **Phase 3:** Predictive model ("we expect 12 deliveries between 09:00–11:00 on Mondays")
- **Future:** Causal model ("when the morning shift arrives, the parking lot fills first, then the lobby, then the production floor")

Semantic Model

What things mean and how they relate. Evolution path:

- **Day 1:** Object class labels (person, vehicle, package) from detection models
- **Phase 2:** Identity resolution and activity classification ("John from maintenance carrying tools")

- **Phase 3:** Relational scene graph ("person A is following person B", "forklift is carrying pallet toward dock 3")
- **Future:** Natural language queryable ("show me every time an unscheduled visitor entered the server room this quarter")

6.3 Uncertainty as a First-Class Citizen

Four mechanisms ensure the system never confuses belief with fact:

- **Confidence scores.** Every assertion carries a confidence: "87% certain zone B is empty." Policies set thresholds: "alert if occupancy confidence exceeds 60% in restricted zone."
- **Coverage maps.** The system knows what it cannot see. "Camera 3 is offline — zone B coverage dropped from 92% to 41%. Switching to cautious mode."
- **Staleness tracking.** Every fact has an age. "Last confirmed empty: 47 seconds ago." Stale facts decay in confidence. Policies can require freshness: "only allow robot entry if zone confirmed empty within 10 seconds."
- **Conflicting signals.** When sensors disagree, the model holds both hypotheses and escalates if the conflict exceeds a threshold. "Motion sensor says occupied, camera says empty — possible occlusion. Confidence: 55% occupied."

6.4 Sensor Fusion Architecture

The world model depends on fusing observations from heterogeneous sources (cameras, motion sensors, door contacts, badge readers) into a coherent state. Fusion is one of the system's hardest technical challenges and follows a phased approach:

Phase 1 (MVP): Rule-based fusion with priority weighting

- Each zone maintains an observation queue from all contributing sensors.
- Observations are merged using weighted rules: camera detections carry higher weight than motion sensors for occupancy; door contacts are authoritative for entry/exit events.
- When sources conflict (motion sensor says occupied, camera says empty), the system applies a configurable conflict resolution policy per zone type: safety zones trust the more cautious source; operational zones trust the higher-weight source.
- Expected accuracy: zone-level occupancy (occupied/empty) correct > 85% of the time in well-covered zones (≥ 2 cameras + supplementary sensors).

Phase 2: Probabilistic fusion with Bayesian state estimation

- Replace rule-based fusion with a lightweight Bayesian filter that maintains a probability distribution over zone states.

- Each sensor observation updates the prior distribution based on learned sensor reliability profiles.
- Enables principled uncertainty quantification: confidence scores become true posterior probabilities rather than heuristic weights.

Phase 3+: Learned fusion models

- Train neural fusion models on deployment-specific data to capture complex sensor correlations (e.g., camera occlusion patterns correlated with time-of-day).

6.5 Tiered World Model Replication

Replica	Content	Purpose
Edge replica	Minimal: zone boundaries, basic occupancy, local sensor state. Updated opportunistically.	Local decisions when connectivity is lost. Edge processors can continue basic detection and alerting.
Facility primary	Authoritative: all on-premise observer fusion, full spatial/temporal/semantic models.	Serves the coding engine, policy engine, and local context API. Must never go down.
Cloud aggregate	Fleet-wide: merged models from multiple facilities, long-term patterns, expensive model results.	Cross-facility governance, fleet-wide anomaly detection, foundation model inference.

6.6 Implementation Approach and Technology Stack

The world model is not a single off-the-shelf component — it is assembled from mature technologies, each solving a distinct sub-problem. No exotic infrastructure is required for the MVP.

Concrete example of world model state. To make the world model tangible: if you freeze time and query it, you get structured beliefs, not raw video. For a zone called "Loading Dock," the state might read: occupancy 2 persons and 1 forklift, confidence 0.87, last confirmed 2 seconds ago, 3 cameras active covering 93% of the zone, with a known blind spot behind the NW shelving unit (approximately 4 square meters). When a camera goes offline, the state doesn't disappear — confidence decays from 0.87 toward zero over time, while boundary sensors (doors, badge readers) provide compensating evidence that supports or contradicts the decaying belief.

State store (current beliefs). Redis serves as the real-time state store. Each zone is a hash with fields for occupancy, confidence, last_confirmed, and coverage. Sub-millisecond reads ensure the policy engine and context API respond instantly. Redis pub/sub propagates state changes to all consumers. Confidence decay is computed on read: `current_confidence =`

`original_confidence × exp(−staleness / halflife)`, so stale beliefs degrade automatically without background processing.

Temporal store (event history). TimescaleDB (PostgreSQL extension for time-series data) stores every observation and state change as a time-series event. This enables historical queries: "average occupancy of zone B between 14:00–16:00 on Mondays this month." TimescaleDB handles retention, downsampling, and compression natively.

Spatial model. Zones are stored as GeoJSON polygons. Shapely (Python computational geometry library) handles all spatial operations: "is this detection inside zone B?", "what percentage of zone B is covered by active cameras?" (camera frustum polygon intersected with zone polygon, divided by zone area). NetworkX models zone topology as a graph — zone A connects to zone B via corridor C. This powers boundary reasoning: if no exit event fired on any connected zone boundary, the belief that an occupant is still present strengthens.

Sensor fusion (Phase 1). Rule-based weighted fusion, implemented in Python. Camera detections carry higher weight (0.8) than motion sensors (0.5); door contact events are near-authoritative (0.95) for entry/exit counting. When sources conflict, a configurable policy resolves per zone type: safety zones trust the more cautious source, operational zones trust the higher-weight source. This is straightforward engineering — not ML — and is debuggable and testable.

Multiple model views. The security, operational, and safety models are not separate AI systems. They are different evaluation functions over the same underlying state store — analogous to database views. Each applies different filtering (security cares about identity; safety ignores identity but watches proximity to hazards), different thresholds (safety triggers at lower confidence), and different defaults under uncertainty (safety assumes occupied; operations assumes normal). These are implemented as Python classes with configurable parameters, not as separate infrastructure.

Sub-problem	MVP Technology	Rationale
State store	Redis	Sub-ms reads, pub/sub, TTL for staleness
Temporal store	TimescaleDB	Time-series queries, retention, compression
Sensor fusion	Custom Python, rule-based	Simple, debuggable, sufficient for MVP
Spatial model	Shapely + PostGIS + NetworkX	Polygon geometry, zone topology graph
Detection pipeline	YOLO/ultralytics	Structured detections with confidence

Model views	Python application layer	Different evaluation functions, same data
Change propagation	Redis pub/sub or NATS	Policy engine and Context API subscribe

Phase 2+ evolution. Bayesian filtering (using libraries like filterpy) replaces rule-based fusion to produce true posterior probabilities instead of heuristic confidence scores. For the full digital twin vision (Phase 5), NVIDIA Omniverse with OpenUSD provides 3D spatial models, physics simulation, and rendering — but this is infrastructure that slots into the same world model interface without upstream changes.

7. Coding Engine

The coding engine is the system's developer experience layer — the component that makes Visual OS programmable by semi-technical operators. It is the direct analog of Claude Code's QueryEngine, but instead of operating on files and functions, it operates on the domain entities defined in Section 5.

Target Operator Persona

The "semi-technical operator" is a facilities manager, security director, or operations lead who understands their physical environment deeply but does not write code. They are comfortable with conditional logic ("if X then Y"), understand concepts like zones and schedules, and can interpret confidence percentages. They typically manage policies today through spreadsheets, radio protocols, and static VMS rule screens. They should not need to understand ML model internals, API design, or infrastructure topology to use the coding engine. This persona should be validated with 3–5 real prospective users before finalizing the coding engine UX.

7.1 How It Works

An operator expresses intent in domain terms. The coding engine translates that intent into validated, permission-gated operations on entities:

Example: Operator Intent → Entity Operations

Operator says: "When a person is detected in the restricted zone after hours, save 30 seconds of pre-event footage to cloud, alert the security team, and if no response in 5 minutes, trigger the alarm." The coding engine translates this into:

1. `stream.getCamera('restricted-zone').subscribe(event='person_detected', timeFilter='after_hours')`
2. `stream.getBuffer(camera, seconds=30).store(tier='cloud', tag='security-event')`
3. `notification.send(group='security-team', priority='high', context=event)`
4. `timer.start(300, onExpire=alarm.trigger(zone='restricted'))`

7.2 LLM Dependency and Fallback

The coding engine's NL parsing relies on a hosted LLM (Claude API or equivalent) to interpret operator intent and map it to entity operations.

- **LLM provider:** Claude API (primary), with the option for on-premise LLM inference at the facility tier for customers with data residency requirements.
- **Accuracy expectations (Phase 1):** Because the NL engine maps to a constrained template library rather than generating arbitrary operation graphs, the expected template-selection accuracy target is > 90% for intents within the supported policy categories. Operators always review the generated operation graph before deployment — the LLM proposes, the human confirms.
- **Fallback on failure:** When the NL engine cannot confidently map an intent to a template (confidence below threshold), the system falls back to a guided wizard that walks the operator through policy creation step-by-step using structured forms. This ensures operators are never blocked by NL parsing failures.
- **Cost model:** LLM API calls occur only during policy authoring (not real-time inference), so cost scales with the number of operator sessions, not with camera count or event volume.

7.3 Cross-Model Queries

The coding engine's most powerful capability is composing queries across multiple world models:

"Alert when the safety model detects presence (confidence > 60%) in a zone the security model marks as restricted, AND the operational model shows no scheduled activity."

This single operator statement queries three models, applies confidence thresholds, and fuses the results into a single decision. The operator didn't need to know about cameras, frame rates, or detection algorithms.

7.4 Degradation-Aware Programming

The coding engine understands the tiered infrastructure and connectivity state. Operators can define behavior across degradation modes:

- **Normal mode:** Full cloud capabilities, foundation model inference, NL queries
- **Degraded mode:** Facility-only, local models, reduced semantic understanding
- **Edge-only mode:** Basic detection and alerting, queued operations for later sync

The coding engine automatically adapts behavior based on the current connectivity state, running the best available version of each policy.

7.5 Permission Model

Inspired by Claude Code's three-mode permission system, adapted for physical-world consequences:

Mode	Behavior	Use Case
Auto-allow	Actions execute immediately when policy conditions are met	Low-risk: logging, notifications, camera preset changes
Ask-human	Action is queued pending human approval via dashboard or mobile	Medium-risk: door locks, robot rerouting, zone alert escalation
Auto-deny	Action is blocked and logged; only humans can override	High-risk: emergency shutdowns, evacuation triggers, access grants

Critically, the confidence threshold interacts with the permission mode. A high-confidence detection might auto-allow an alert, while the same alert at lower confidence triggers ask-human mode. This prevents the system from taking high-impact actions based on uncertain information.

8. Tiered Infrastructure

The system operates across three infrastructure tiers, each with different characteristics. The coding engine, world model, and entity runtime all understand this tiered topology and adapt their behavior accordingly.

8.1 Edge Tier

- **Compute:** Camera-onboard (ARM, low power) or edge appliance (GPU-equipped NUC/Jetson)
- **Storage:** SD card ring buffer (24–72 hours), volatile
- **Capabilities:** Lightweight ML inference (YOLO, MobileNet), motion detection, frame filtering, event tagging
- **Availability:** Always running, no network dependency
- **Decision authority:** Can trigger local alerts, buffer important events, filter non-events before sending upstream

8.2 Facility Tier

- **Compute:** On-premise server(s) with GPU, running the main Visual OS runtime

- **Storage:** NVR with RAID, 7–90 day retention, fast local access
- **Capabilities:** Full pipeline processing, cross-camera tracking, vectorization, local LLM inference (for customers who prohibit cloud), world model primary replica
- **Availability:** High (local network), survives internet outages
- **Decision authority:** Full policy evaluation, action dispatch, human interface, coding engine execution

8.3 Cloud Tier

- **Compute:** Elastic GPU/TPU for foundation model inference, fleet-wide analytics
- **Storage:** Tiered object storage (hot/cold/archive), indefinite retention
- **Capabilities:** Video foundation models, NL-queryable video archives, cross-facility aggregation, fleet governance dashboard
- **Availability:** Requires internet connectivity; latency 50–500ms
- **Decision authority:** Fleet-wide policies, expensive inference, long-term pattern analysis, compliance reporting

8.4 Capacity Planning

A capacity planning model must be developed as a required MVP deliverable, covering: compute requirements per camera stream at each tier, GPU sizing for the detection pipeline, storage growth rate and retention cost curves, and projected cost-per-camera-per-month at scale. This model informs hardware recommendations for pilot customers and validates the business case for deployments beyond the MVP's 4–8 camera scope.

8.5 Connectivity and Failover

The system uses an explicit degradation graph:

- **All tiers connected:** Full capabilities. Cloud handles expensive inference; facility handles real-time; edge handles filtering.
- **Cloud offline:** Facility operates autonomously. Local models substitute for cloud models at reduced quality. Events queued for cloud sync. NL queries unavailable until reconnection.
- **Facility offline:** Edge devices operate independently. Basic detection and local alerting only. All events buffered on edge storage. World model frozen at last known state.

On reconnection, queued events replay in order. The world model reconciles divergent state. Conflicts are flagged for human review.

9. Security Considerations

Visual OS has security requirements beyond typical software systems because actions have physical-world consequences and the system processes sensitive video data.

9.1 Physical-World Safety

- **False positive cost:** A false alert that locks a building or reroutes a robot fleet has real cost. Confidence thresholds and approval workflows mitigate this.
- **Failsafe defaults:** When the system is uncertain or failing, it defaults to a safe state. A robot operating unsupervised because the monitoring system crashed is an unacceptable failure mode.
- **Adversarial physical environments:** Masks, tailgating, spoofing, intentional camera obstruction. Multi-sensor verification and anomaly detection across the world model reduce single-point-of-deception risk.

9.2 Data Privacy

- **Privacy zones:** Configurable regions where detection is active but no video is recorded or stored (e.g., bathrooms, break rooms).
- **Face redaction:** Policy-configurable automatic redaction in stored footage. Some models need identity; others don't.
- **Data residency:** For customers who cannot send video off-premises, the full pipeline runs at the facility tier with local LLM inference. No video bytes leave the building.
- **Retention policies:** Enforced at the storage tier entity level. Automatic eviction with audit trails.
- **GDPR/local compliance:** The compliance world model tracks what data was collected, when, why, and when it must be deleted. A Data Protection Impact Assessment (DPIA) is a required deliverable before any pilot deployment in EU jurisdictions. The system must support Data Subject Access Requests (DSARs) across all storage tiers, including the ability to locate and export or delete all data associated with a specific individual. The tension between immutable audit trails (Section 9.3) and the right to erasure must be resolved through a documented approach — such as cryptographic erasure (destroying encryption keys) while retaining anonymized audit metadata. Cross-border data transfer rules apply when the cloud tier is hosted outside the customer's jurisdiction; the system must support configurable data residency policies per storage tier. Consent mechanisms (signage-based implied consent, explicit opt-in, or legitimate interest) must be configurable per deployment context.

9.3 System Security

- **Secure communication:** TLS between all tiers. Mutual authentication between edge devices and facility servers.

- **Permission isolation:** The coding engine's permission model prevents unauthorized escalation. Operators cannot grant themselves higher permissions than their role allows.
- **Audit trail:** Every action, every policy change, every approval decision is logged immutably. The compliance world model provides queryable audit history.
- **Supply chain:** Edge devices and facility servers run signed firmware. Model deployments are verified. No unsigned code executes in the pipeline.

10. MVP Definition

MVP Design Philosophy

Any modern VMS can define zones and trigger alerts. The MVP must demonstrate what no existing system can do: maintain a belief about physical reality that degrades gracefully under uncertainty, let operators program that reality through domain entities rather than configuration screens, and serve situational awareness to external systems as infrastructure. Every demo moment should make a stakeholder say "I've never seen a video system do that."

10.1 The Five "Wow Moments"

The MVP is structured around five demonstration scenarios, each proving a core architectural principle. These are not features — they are proof points that Visual OS operates on fundamentally different assumptions than existing video monitoring systems.

Wow Moment 1: The System Knows What It Doesn't Know

Principle demonstrated: Partial observability and uncertainty as first-class citizens.

Scenario: During a live demo, an operator deliberately disconnects a camera. In a traditional VMS, that camera's feed goes black and the corresponding zone becomes a blind spot. In Visual OS, the dashboard shows something no existing system shows:

- **Before disconnect:** "Zone B: 2 persons detected. Confidence: 91%. Coverage: 3 cameras active."
- **After disconnect:** "Zone B: 2 persons believed present (decaying confidence: 74% → 68% → ...). Coverage degraded to 58%. Last direct observation: 12 seconds ago. No exit events detected at zone boundaries, supporting continued occupancy belief. Safety model switched to CAUTIOUS mode."

The system doesn't just lose a feed — it reasons about what it still knows, what it's losing confidence in, and what compensating evidence supports its current belief. The confidence score decays over time, and policies automatically adjust their behavior based on the degraded certainty. This is the single most differentiated capability: the system models its own ignorance.

Wow Moment 2: Programming Reality, Not Configuring Software

Principle demonstrated: Entity-oriented coding engine.

Scenario: An operator describes what they want in natural language or structured domain terms. Not "configure rule: camera 3, motion threshold 0.6, webhook URL" but:

"The loading dock should have no more than 3 people during off-hours. If a fourth person arrives, give the operational model 30 seconds to confirm they're an expected late-shift worker. If unconfirmed, alert the floor supervisor and save 2 minutes of context footage."

The coding engine translates this into entity operations: it resolves "loading dock" through the spatial model, subscribes to the security model's occupancy tracker, queries the operational model's shift schedule for confirmation, configures a 30-second timer with a conditional branch, and wires up the alert and storage actions with proper retention tags. The operator sees the generated entity operation graph and can adjust thresholds, but they never wrote a line of code or navigated a configuration wizard.

What makes this different from existing "rule builders": the coding engine understands the relationships between entities. It knows which cameras observe the loading dock, what the operational model's shift data looks like, and what storage tier to use for event footage. It validates the entire operation chain before deployment and flags conflicts with existing policies.

Wow Moment 3: Same Scene, Different Truths

Principle demonstrated: Multiple partial world models with different purposes.

Scenario: A person walks through the facility. The dashboard shows three simultaneous interpretations of the same physical event:

- **Security model:** "Unidentified individual in corridor B. No badge event. Confidence: 82%. Risk level: ELEVATED."
- **Operational model:** "Additional person in corridor B. Foot traffic is 15% above baseline for this hour. Workflow impact: NONE."
- **Safety model:** "Human presence confirmed near machinery zone perimeter. Minimum safe distance: MAINTAINED. Status: CLEAR."

Three models, three different assessments, from the same sensor data. The security model is concerned because there's no identity match. The operational model is unconcerned because foot traffic is within normal range. The safety model is watching proximity to hazards. Each model has its own error bias, confidence calculation, and consumer. The operator can see all three perspectives simultaneously and understand why the system is recommending a specific action.

This demonstrates that Visual OS doesn't give a single "answer" — it gives structured situational awareness from multiple governance perspectives, letting the right consumer (security team, ops manager, safety system) see the interpretation that matters to them.

Wow Moment 4: Infrastructure, Not Application

Principle demonstrated: Context API for autonomous agents and external systems.

Scenario: An autonomous agent queries the Visual OS context API. The MVP targets at least one real agent integration (e.g., a robotic vacuum, a delivery robot, or a warehouse AGV querying zone occupancy) alongside a simulated multi-agent scenario for demonstration breadth. Example query:

- **Agent query:** "Is corridor C clear for transit? I am a cleaning robot with dimensions 0.6m x 0.4m."
- **Visual OS response:** "Corridor C: 1 person detected at coordinates (x, y), moving eastbound at ~1.2 m/s. Estimated clearance time: 8 seconds. Confidence: 87%. Suggested action: WAIT or REROUTE via corridor D (currently empty, confidence: 94%, last confirmed: 3 seconds ago). Coverage note: eastern segment of corridor C has 71% coverage due to camera occlusion by shelving unit."

The response includes not just the answer, but confidence, staleness, alternative routes, and coverage limitations. This demonstrates that Visual OS is not a screen for humans to watch — it is infrastructure that other systems depend on. The context API is the product's primary interface for the autonomous future, and the MVP proves it works.

The combination of a real agent integration (validating product-market fit) and a simulated multi-agent scenario (demonstrating architectural breadth) proves to stakeholders that Visual OS is positioned to become essential infrastructure for every autonomous physical operation.

Wow Moment 5: Graceful Degradation in Real Time

Principle demonstrated: Facility-tier resilience with transparent capability management.

Scenario: During the demo, the operator simulates a cloud connectivity loss. Instead of the system failing or showing errors:

- **Dashboard shows:** "Connectivity: CLOUD OFFLINE. Switching to facility-only mode. Affected capabilities: NL video queries (queued), cloud archival (buffering locally, 4.2 GB queued). Unaffected: real-time detection, local policies, alerting, context API, world model updates."
- **All existing policies continue:** Detection, alerting, and the context API operate normally using the facility-tier world model and local inference.

- **On reconnection:** Queued events sync to cloud. World model reconciles. The operator sees a brief "sync complete" notification. Nothing was lost.

This demonstrates that Visual OS is designed for the real world, where connectivity is not guaranteed. In the MVP, facility-tier degradation (surviving cloud outages) is a day-one capability. Full edge-tier autonomy (edge devices operating independently during facility outages) is delivered in Phase 2 with real edge hardware deployment.

> **Scope note:** The MVP demonstrates cloud↔facility degradation with real infrastructure. Edge-tier autonomy is simulated in the MVP demo to illustrate the architectural direction, but production edge deployment requires Phase 2 hardware integration and testing.

10.2 MVP Technical Scope

The five wow moments above require the following technical components:

Entity Runtime

- **Camera entity:** Ingest 4–8 RTSP streams with health monitoring and automatic failover detection
- **Stream entity:** Route, downsample, subscribe to events, buffer pre-event footage
- **Storage entity:** Facility-tier with configurable retention and event-tagged archival
- **Pipeline entity:** Detection pipeline producing structured events with bounding boxes and confidence scores
- **Connectivity entity:** State tracking for edge/facility/cloud tiers with automatic degradation mode switching

World Model

- **Spatial:** 2D floor plan with polygon zones, camera-to-zone mapping, zone topology (which zones connect to which)
- **Temporal:** Event log with timestamps; basic pattern baseline ("normal occupancy for this zone at this hour")
- **Semantic:** Object class labels (person, vehicle, package) with per-detection confidence scores
- **Uncertainty engine:** Confidence decay over staleness, coverage maps recalculated on camera state changes, conflicting signal detection
- **Multiple model views:** Security, operational, and safety interpretations of the same underlying observations, each with distinct error bias

Coding Engine

- **Natural language intent parsing:** Operator describes a policy in natural language; the engine generates an entity operation graph for review and deployment.
  Phase 1 scope: The NL engine maps operator intent to a curated library of policy templates (occupancy rules, access restrictions, schedule-based alerts, event-triggered recording) and fills template parameters from the operator's description. This constrains the output space to validated, tested operation patterns while still providing an NL interface.
  Phase 2 scope: Free-form NL → arbitrary entity operation graph generation, supporting novel policy types beyond the template library.
- **Entity-aware validation:** The engine understands zone topology, camera coverage, available models, and existing policies
- **Cross-model queries:** Policies can reference multiple world models with per-model confidence thresholds
- **Degradation-aware rules:** Policies include fallback behavior for reduced connectivity modes

Context API

- **Query endpoint:** External systems can query zone state, occupancy, clearance, and routing suggestions
- **Structured responses:** Every response includes confidence, staleness, coverage quality, and suggested actions
- **Subscription endpoint:** External systems can subscribe to world model changes for specific zones or event types

Human Interface

- **World model dashboard:** Floor plan showing live zone states, confidence heat map, coverage overlay, active detections
- **Multi-model view:** Toggle between security, operational, and safety interpretations of the same scene
- **Coding engine interface:** Natural language policy input with generated operation graph visualization
- **Degradation status:** Real-time infrastructure health showing tier connectivity, capability state, and queued operations

10.3 Success Criteria

What the MVP Proves to Stakeholders

1. **WORLD MODEL:** The system maintains structured beliefs about physical reality — not raw detections — with confidence, staleness, and coverage awareness. When information is lost, the system explicitly models its growing uncertainty rather than silently failing.
2. **MULTIPLE PERSPECTIVES:** The same physical scene is interpreted differently by security, operational, and safety models, each optimized for different governance concerns with different error tolerances. No existing system does this.
3. **PROGRAMMABLE REALITY:** Operators express governance intent in domain terms and the coding engine translates it into validated entity operations. The gap between "what I want" and "how to configure it" collapses — the Claude Code moment for physical-world governance.
4. **INFRASTRUCTURE, NOT APPLICATION:** The context API serves structured situational awareness to autonomous agents, proving the system's positioning as essential infrastructure for the autonomous future, not just another camera viewer.
5. **REAL-WORLD RESILIENCE:** The system degrades transparently and recovers gracefully across connectivity disruptions, proving it is designed for production deployment in environments where reliability is non-negotiable.

10.4 What the MVP Explicitly Does Not Include

To maintain focus and deliver within 4 months, the MVP defers:

- **Identity resolution:** No face recognition, badge correlation, or person re-identification across cameras. The semantic model uses anonymous object classes only.
- **Cross-camera tracking:** Objects are tracked per-camera. Cross-zone tracking via exit/entry correlation at zone boundaries only, not continuous re-identification.
- **Cloud tier:** The MVP operates entirely at the facility tier. Cloud archival and foundation model inference are Phase 2.
- **Edge tier autonomy:** Edge processing is simulated for the degradation demo. Real edge deployment is Phase 2.
- **NL video queries:** Querying video history in natural language requires vectorization infrastructure deferred to Phase 2.

These exclusions are intentional — they are deferred to later phases, not abandoned. The world model interface is designed to accept these capabilities when they arrive without architectural changes.

11. Development Roadmap

Phase 1: Foundation (Months 1–4)

Entity runtime + world model with uncertainty + coding engine + context API + human interface. Proves the five wow moments: the system models its own ignorance, operators program in

domain terms, multiple models interpret the same scene, external agents consume situational awareness, and the system degrades transparently.

- **Deliverable:** Working system monitoring 4–8 cameras with world-model-centric governance, multi-perspective interpretation, NL policy authoring, and a live context API
- **World model:** Zones + occupancy + confidence + staleness + coverage maps + multi-model views (security/operational/safety)
- **Users:** Internal testing, 1–2 pilot customers, investor demo-ready

Phase 2a: Intelligence (Months 4–6)

AI-powered scene understanding, temporal pattern learning, multi-camera tracking. Edge tier hardware deployment and testing.

- **Deliverable:** System learns normal patterns and flags anomalies; real edge devices running lightweight inference
- **World model:** Adds object tracking, activity classification, pattern baselines
- **Infrastructure:** Edge tier deployed with real hardware; tiered storage with cost optimization
- **Users:** 3–5 pilot customers in initial verticals

Phase 2b: Advanced Authoring + Cloud (Months 6–8)

Free-form NL policy authoring (extending Phase 1's template-based NL to support arbitrary policy generation). Cloud tier standup.

- **Deliverable:** Operators author policies in free-form natural language; cloud archival and foundation model inference available
- **Infrastructure:** Cloud tier deployed; cross-facility data aggregation pipeline
- **Users:** 5–10 pilot customers across different verticals

Phase 3: Platform (Months 8–14)

Multi-agent orchestration, integration protocol, context API for autonomous agents, cross-camera intelligence.

- **Deliverable:** Third-party systems (robots, BMS, access control) consume the context API; zone agents coordinate autonomously
- **World model:** Adds semantic scene graphs, identity resolution, cross-zone tracking
- **Infrastructure:** Fleet-wide aggregation for multi-facility customers
- **Users:** 20–50 customers; first enterprise accounts with autonomous agent integration

Phase 4: Autonomous Governance (Months 14–20)

Proactive monitoring, background pattern analysis, predictive world model, NL video querying, full compliance framework.

- **Deliverable:** System proactively suggests policies, predicts incidents, and provides full audit trails for regulators
- **World model:** Adds predictive temporal model, causal reasoning, NL-queryable video archive
- **Infrastructure:** Foundation model integration; on-premise LLM option for data-sovereign customers
- **Users:** 100+ customers; regulatory certification for specific verticals (healthcare, manufacturing, logistics)

Phase 5 and Beyond: Digital Twin (Months 20+)

Full physics-aware digital twin. Simulation capabilities ("what if we rearrange the warehouse?"). Multi-facility fleet governance. Industry-specific modules.

12. Resourcing and Dependencies

A resourcing plan must be developed alongside the technical architecture to validate the MVP timeline. The following areas require definition:

- **Team composition and roles:** Engineering disciplines required (backend, ML/CV, frontend, infrastructure), product and design involvement, and any specialized expertise gaps.
- **Headcount vs. scope validation:** An honest assessment of whether the available team can deliver the MVP scope within 4 months, with scope reduction as the preferred lever if resourcing falls short.
- **External dependencies:** Third-party services (LLM APIs, Nx Meta platform components), hardware procurement (facility-tier GPUs), and pilot customer site access.
- **Infrastructure and operational budget:** Compute, storage, API costs, and development environment provisioning.
- **Hiring risk:** Identification of any unfilled roles that are critical-path blockers for the MVP timeline.

13. Competitive Positioning

Dimension	Traditional VMS (Milestone, Genetec)	AI Video Analytics (BriefCam,	Emerging AI-Native (Ambient.ai)	Visual OS

		Verkada, Rhombus)		
Core function	Record and playback	Detect and alert	Contextual threat detection	Govern and orchestrate
User	Security guard	Security analyst	Security analyst	Operations manager (semi-technical)
World model	None (human memory)	Per-camera analytics	Single-model scene understanding	Multi-model digital twin with uncertainty
Programmability	Static config	Rule templates	Pre-built use cases	Entity-oriented coding engine
Integration	ONVIF/RTSP only	Webhook alerts	Limited API	Context API for autonomous agents
Offline operation	Local recording only	Varies	Cloud-dependent	Tiered degradation with edge autonomy
Positioning	Cost center (security)	Cost center (security)	Security platform	Infrastructure (governance platform)

Key differentiation to validate: Ambient.ai is the closest competitor, offering AI-native scene understanding with contextual awareness. Visual OS's differentiation rests on three capabilities Ambient.ai does not offer: multiple purpose-built world models with explicit uncertainty, an entity-oriented coding engine for operator-defined governance, and a Context API designed for autonomous agent consumption. Each "wow moment" should be validated against Ambient.ai's current and roadmap capabilities.

The key competitive moat is the world model layer combined with the coding engine. Competitors would need to rebuild both the entity model (understanding of the video monitoring domain) and the abstraction layer (making it programmable by semi-technical users). The integration protocol creates a network effect: as more autonomous agents consume the context API, the platform becomes harder to replace.

Appendix A: Glossary

Term	Definition
World Model	The system's continuously updated representation of physical reality, maintained through sensor fusion and AI inference

Entity Runtime	The infrastructure layer that manages the lifecycle of streams, storage, pipelines, models, and connectivity
Coding Engine	The developer experience layer that translates operator intent into validated operations on domain entities
Context API	The interface through which autonomous agents and external systems query the world model for situational awareness
Policy Engine	The component that continuously evaluates governance rules against world model state
Observer	Any data source that contributes observations to the world model (cameras, sensors, robots, access systems, humans)
Zone	A named spatial region with semantic properties and governance rules
Confidence Score	A 0–1 value attached to every world model assertion, representing the system's certainty
Staleness	The time since a world model assertion was last confirmed by direct observation
Degradation Graph	The explicit mapping of what capabilities are available at each connectivity state
Digital Twin	The long-term vision of the world model: a physics-aware simulation of the physical space